PES UNIVERSITY, RR CAMPUS

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AIML)

LAB 6: CODE AND BRANCH COVERAGE ANALYSIS

Dynamic White-Box Testing, Equivalence Partitioning & Decision Branch Coverage

Student: Dharshini V | SRN: PES1UG24AM428 | Course: Software Engineering Lab

Assigned Repo: SETAPESU26/L6_L7_56_course_registration | Personal Repo: DharshiniPES/L6_L7_56_course_registration

1. Executive Summary & Theoretical Foundations

Software testing completeness cannot be judged solely by the number of passing unit tests; it requires rigorous structural measurement of how thoroughly the test suite exercises the underlying code base. In this laboratory, coverage analysis was conducted on an assigned Course Registration module (course_registration.py from repository SETAPESU26/L6_L7_56_course_registration) using pytest and Python's coverage library.

Coverage Metric	Mathematical Definition / Concept	Practical Engineering Purpose
Line (Statement) Coverage	Percentage of executable program statements traversed at least once during test execution.	Identifies completely unreached functions, dead code blocks, and unexecuted error handlers.
Branch Coverage	Percentage of decision outcomes (both True and False branches of every control gate) evaluated.	Forces validation of alternate paths and boundary gates, eliminating false confidence.
Path Coverage	Percentage of all possible unique execution pathways from function entry to exit traversed.	Exhaustive coverage of every permutation of combined decision conditions.
Function Coverage	Percentage of defined functions or methods invoked at least once during the test suite.	High-level smoke assessment verifying that all entry points are callable.

CRITICAL TAKEAWAY: Why Line Coverage Gives a False Sense of Security

Line coverage simply asks: 'Did execution pass through this line at least once?' When an if-condition evaluates to True, the statements inside execute and report 100% line coverage. However, the alternate branch (where the condition evaluates to False) may remain completely untested! Branch coverage guarantees that every conditional branch is tested in BOTH directions (True and False), preventing critical logic and boundary bugs from slipping into production.

2. Architecture of the System Under Test (`course_registration.py`)

The course_registration.py module provides three core academic governance functions governed by credit policies, seat quotas, and late fee schedules:

Function Name	Parameters	Business Rules & Decision Logic	Equivalence Classes
can_register()	student_status, current_credits, course_credits, prerequisites_met	Validates status against MAX_CREDITS (regular: 24, probation: 16, honors: 28); rejects invalid credits (<= 0); checks prerequisite gating; enforces credit caps.	Status: {regular, probation, honors, invalid}; Credits: {valid, <=0}; Prereq: {True, False}; Limit: {<=max, >max}
waitlist_position()	enrolled, capacity, waitlist_length	Rejects non-positive capacity (<= 0); returns 0 if seats remain (enrolled < capacity); caps waitlist at 10 (returns -1 if length >= 10); calculates position as waitlist_length + 1.	Capacity: {<=0, >0}; Seat State: {open, full, over-enrolled}; Waitlist: {empty(0), normal(1-9), full(10), overflow(>10)}
late_fee()	days_late, credits, has_waiver	0 fee if on-time (days_late <= 0) or	Days: {<=0, 1-7, >7}; Waiver:

	has_waiver is True. Week 1 (1-7 days) base fee = ₹500. Beyond 7 days: ₹500 + (days_late - 7)*100. Overload surcharge (+₹250 if credits > 20). Absolute cap at ₹2000.	{True, False}; Overload: {<=20, >20}; Cap State: {uncapped(<2000), exact(=2000), capped(>2000)}

3. Baseline Analysis (Starter Test Suite)

Initial execution of the starter test_course_registration.py revealed substantial coverage deficits. With only two test cases, more than 60% of the program's statements and 68% of its decision branches were unreached.

Coverage Metric	Starter Result	Target Rubric	Deficit / Missing Coverage
Statement (Line) Coverage	38% (12/32 statements)	>= 90% (Achieved 100%)	20 lines completely unexecuted (lines 6, 8, 10, 12, 18, 21-23, 27-39)
Branch Coverage	32% (8/24 branch paths)	>= 90% (Achieved 100%)	18 branch paths unreached or only partially traversed
Test Case Suite Size	2 test cases	Comprehensive	Lacked exception testing, boundary checking, and waiver handling

4. Test Design Matrix (Equivalence Partitioning & Boundary Value Analysis)

To satisfy the lab requirements and eliminate all untested paths, 54 automated test cases were designed across two distinct test suites (CodeCoverageTest_course_registration.py and BranchCoverageTest_course_registration.py).

Test ID	Target Function	Tested Decision Branch / Condition	Input Parameters	Expected Result
TC-REG-01	can_register()	student_status not in MAX_CREDITS (True)	('exchange', 10, 4, True)	raises ValueError('unknown status')
TC-REG-02	can_register()	student_status in MAX_CREDITS ('regular', False)	('regular', 12, 4, True)	'registered'
TC-REG-03	can_register()	student_status in MAX_CREDITS ('probation', False)	('probation', 10, 4, True)	'registered'
TC-REG-04	can_register()	student_status in MAX_CREDITS ('honors', False)	('honors', 20, 4, True)	'registered'
TC-REG-05	can_register()	course_credits <= 0 (True, boundary: 0)	('regular', 12, 0, True)	raises ValueError('invalid course credits')
TC-REG-06	can_register()	course_credits <= 0 (True, negative: -3)	('regular', 12, -3, True)	raises ValueError('invalid course credits')
TC-REG-07	can_register()	course_credits <= 0 (False, positive: 2)	('regular', 10, 2, True)	'registered'
TC-REG-08	can_register()	not prerequisites_met (True)	('regular', 12, 4, False)	'prerequisites missing'
TC-REG-09	can_register()	not prerequisites_met (False)	('regular', 12, 4, True)	'registered'
TC-REG-10	can_register()	current + course > limit (True, regular: 25 > 24)	('regular', 21, 4, True)	'credit limit exceeded'
TC-REG-11	can_register()	current + course <= limit (False, boundary: 24 == 24)	('regular', 20, 4, True)	'registered'
TC-REG-12	can_register()	current + course > limit (True, probation: 18 > 16)	('probation', 14, 4, True)	'credit limit exceeded'

TC-REG-13	can_register()	current + course <= limit (False, boundary: 16 == 16)	('probation', 12, 4, True)	'registered'
TC-REG-14	can_register()	current + course > limit (True, honors: 30 > 28)	('honors', 26, 4, True)	'credit limit exceeded'
TC-REG-15	can_register()	current + course <= limit (False, boundary: 28 == 28)	('honors', 24, 4, True)	'registered'
TC-WL-01	waitlist_position()	capacity <= 0 (True, boundary: 0)	(10, 0, 0)	raises ValueError('invalid capacity')
TC-WL-02	waitlist_position()	capacity <= 0 (True, negative: -5)	(10, -5, 0)	raises ValueError('invalid capacity')
TC-WL-03	waitlist_position()	capacity <= 0 (False, positive: 30)	(15, 30, 0)	0
TC-WL-04	waitlist_position()	enrolled < capacity (True, seats open: 25 < 30)	(25, 30, 2)	0
TC-WL-05	waitlist_position()	enrolled < capacity (False, exact boundary: 30 == 30)	(30, 30, 0)	1
TC-WL-06	waitlist_position()	enrolled < capacity (False, over-enrolled: 35 > 30)	(35, 30, 3)	4
TC-WL-07	waitlist_position()	waitlist_length >= 10 (True, boundary: 10)	(30, 30, 10)	-1 (waitlist closed)
TC-WL-08	waitlist_position()	waitlist_length >= 10 (True, overflow: 15)	(30, 30, 15)	-1 (waitlist closed)
TC-WL-09	waitlist_position()	waitlist_length >= 10 (False, boundary: 9)	(30, 30, 9)	10 (assigned position)
TC-WL-10	waitlist_position()	waitlist_length >= 10 (False, empty: 0)	(30, 30, 0)	1 (assigned position)
TC-FEE-01	late_fee()	days_late <= 0 (True, boundary: 0)	(0, 15, False)	0 (no late fee)
TC-FEE-02	late_fee()	days_late <= 0 (True, negative days: -3)	(-3, 15, False)	0 (no late fee)
TC-FEE-03	late_fee()	days_late <= 0 (False, positive: 1)	(1, 15, False)	500
TC-FEE-04	late_fee()	has_waiver (True)	(10, 24, True)	0 (waiver honored)
TC-FEE-05	late_fee()	has_waiver (False)	(5, 15, False)	500 (fee applied)
TC-FEE-06	late_fee()	days_late <= 7 (True, middle: 4 days)	(4, 15, False)	500
TC-FEE-07	late_fee()	days_late <= 7 (True, boundary: 7 days)	(7, 15, False)	500
TC-FEE-08	late_fee()	days_late <= 7 (False, boundary: 8 days -> 500+100)	(8, 15, False)	600
TC-FEE-09	late_fee()	days_late <= 7 (False, multi-day: 12 days -> 500+500)	(12, 15, False)	1000
TC-FEE-10	late_fee()	credits > 20 (False, boundary: 20 credits)	(5, 20, False)	500 (no surcharge)
TC-FEE-11	late_fee()	credits > 20 (False, standard: 12 credits)	(5, 12, False)	500 (no surcharge)
TC-FEE-12	late_fee()	credits > 20 (True, boundary: 21 credits -> +250)	(5, 21, False)	750 (surcharge applied)
TC-FEE-13	late_fee()	credits > 20 (True, overloaded: 24 credits -> +250)	(8, 24, False)	850 (surcharge applied)
TC-FEE-14	late_fee()	fee > 2000 (False, exact boundary: 500+1500=2000)	(22, 20, False)	2000 (uncapped exact)
TC-FEE-15	late_fee()	fee > 2000 (False, lower: fee = 600 < 2000)	(8, 15, False)	600 (uncapped)

TC-FEE-16	late_fee()	fee > 2000 (True, cap triggered: 2550 -> 2000)	(25, 24, False)	2000 (capped at maximum)

5. Verification Results & Terminal Reports

Both test suites were executed with Python's coverage library to gather line coverage and branch coverage metrics. As documented below, 100% Statement Coverage and 100% Branch Coverage were achieved.

Test Suite File	Stmts	Miss	Branch	BrPart	Coverage	Evaluation Status
CodeCoverageTest_course_registration.py	31	0	0	0	100%	PASS (Exercises 100% executable lines)
BranchCoverageTest_course_registration.py	92	0	0	0	100%	PASS (Evaluates 100% branch decisions)
course_registration.py (Under Test)	32	0	24	0	100%	PERFECT (32/32 statements, 24/24 branches)
TOTAL Combined Program & Tests	124	0	24	0	100%	RUBRIC EXCEEDED (Target >= 90%)

TERMINAL EVIDENCE: Figure 1: Code (Line) Coverage Terminal Output

[COMMAND TO RUN]

```
coverage run -m pytest CodeCoverageTest_course_registration.py
coverage report -m
```

[VERIFIED TERMINAL OUTPUT CAPTURE]

```
Name                                      Stmts   Miss  Cover   Missing
-----------------------------------------------------------------------
CodeCoverageTest_course_registration.py      31      0   100%
course_registration.py                       32      0   100%
-----------------------------------------------------------------------
TOTAL                                        63      0   100%
```

TERMINAL EVIDENCE: Figure 2: Branch Coverage Terminal Output

[COMMAND TO RUN]

```
coverage run --branch -m pytest BranchCoverageTest_course_registration.py
coverage report -m
```

[VERIFIED TERMINAL OUTPUT CAPTURE]

```
Name                                      Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------------------------
BranchCoverageTest_course_registration.py     92      0      0      0   100%
course_registration.py                        32      0     24      0   100%
-----------------------------------------------------------------------------------------
TOTAL                                        124      0     24      0   100%
```

TERMINAL EVIDENCE: Figure 3: Git Commit History (Minimum 2 Commits)

[COMMAND TO RUN]

```
git log --oneline -n 4
```

[VERIFIED TERMINAL OUTPUT CAPTURE]

```
4ce85bb (HEAD -> main) Add BranchCoverageTest_course_registration.py achieving 100% branch coverage, reports and documentation
9846edc Add CodeCoverageTest_course_registration.py achieving 100% statement (line) coverage
71e1df4 push lab 7
d04bf92 commit lab6
```

6. Version Control Deliverables & Commits

Commit Hash	Phase / Stage	Commit Message Summary	Key Deliverables & Changes
9846edc	Stage 1: Code Coverage	Add CodeCoverageTest_course_registration.py achieving 100% statement (line) coverage	Added 13 statement-focused tests exercising all lines in course_registration.py; configured .gitignore.
4ce85bb	Stage 2: Branch Coverage	Add BranchCoverageTest_course_registration.py achieving 100% branch coverage, reports and documentation	Added 41 comprehensive branch, boundary, and equivalence tests; generated text reports; updated README.md.